

QUDA 与 PyQCU 分布式 Strict MultiGrid: fine/compact/full halo、R/P coarse halo 与 distributed fused FGMRES

PyQCU 工程分析记录

2026 年 9 月 18 日

摘要

本报告记录 PyQCU 把 Strict MultiGrid 从“单 rank 原型”补齐为真正分布式实现的过程与验证结果,并在统一协议下与 QUDA 做正确性和性能对照。实现的四个核心部件是: fine/compact/full 三个层级的 face halo、 P/R 粗层 halo、分布式 fused FGMRES (全局内积与全局范数), 以及分布式的 Galerkin setup (分布式 roll 与 halo 化的 fine 算子)。同时修正了最细层“总迭代次数”的记录口径, 使其与 QUDA 的外层 GCR/FGMRES `total_iter` 语义一致。正确性用 1/2/4 rank 与单 rank 全局参考逐分量对比, 粗层 full/compact 算子与分布式 setup 资产都达到机器精度; 性能矩阵按 cold + 2 warmup + 5 steady、c64/c128、三种格点、trace 开关、MG-1/2/3 层、V100 单卡与双 P100 组合采集, 缺失组合在报告中显式标注。

目录

1 范围、结论与证据等级

1.1 本轮交付

- 运行期: fine/compact/full halo、 P/R 粗层 halo、分布式 fused FGMRES 全部落地, 并在 1/2/4 rank、四种分解 (x 、 t 、 $x-t$ 、 $x-y$)、c64/c128 上验证到机器精度或单精度舍入水平。
- setup: 粗层 $X/Y/\hat{Y}$ 的构造改为分布式 roll 加 halo 化 fine 算子, 2/4 rank 的 rank 边界点与内部点都与单 rank 全局参考一致。
- 口径: 最细层“总迭代次数”取外层 (右预条件) FGMRES 的 `total_iter`; pre/post smoother、restriction/prolongation 与 coarse solver 的迭代数分开记录, 不再混入总迭代。
- 工具: 完整笛卡尔矩阵编排器、收集器扩展、trace 解析与报告生成器 (CSV/SVG/PDF/LaTeX) 全部入库。

1.2 结论摘要

- 正确性（本轮实测，E1）：分布式 full/compact 粗层算子与全局参考在 c128 上一致到 10^{-16} ；分布式 setup 资产在 2/4 rank、2/3 层、site-batch/colored 两种模式下与单 rank 全局参考一致；分布式求解真残差 $< 10^{-6}$ 。
- 性能（V100 单卡、c64、MG-2/3）：PyQCU 的 steady 中位时间在 6 个有效 case 上全部低于 QUDA (0.12–0.88 倍)，外层迭代数为 QUDA 的 0.24–0.53 倍，单次迭代成本为 1.8–6.0 倍。
- 性能（V100 单卡、c128、MG-2/3、trace off）：在小格与中格的 4 个可比 case 上，PyQCU steady 中位时间为 QUDA 的 0.091–0.512 倍，即 QUDA-over-PyQCU 比值为 1.953–10.965 倍；大格 QUDA 本轮矩阵采集受容量限制，未并入正式比值。
- 口径（本轮修正，E1）：最细层“总迭代次数”统一为外层 FGMRES/GCR 迭代数，并被写入 JSON 的 `iteration\_semantics`。
- 缺口（E1，未填充）：MG-1 两侧都不可用；QUDA c128 大格单元在当前矩阵编排器下为 device OOM；双 P100 的 QUDA 侧缺 `sm\_60` 二进制。三者都以 blocked/failed 记录，未使用替代实现或外推数值。

1.3 证据等级

表 1: 本报告的证据分级

等级	含义
E1	本机实测；命令、版本与产物路径可复现
E2	源码级推导或与 QUDA 源码逐行对照，未在本轮重跑
E3	文献或历史报告结论，只做引用

除特别标注外，本报告的正确定性结论均为 E1；QUDA 语义结论为 E2。

2 分布式 Strict-MG 的实现

2.1 为什么原实现不是真正的分布式

原 strict 路径把 fine/coarse 的每一个算子都写成“本地周期格点”形式：所有 $\pm\mu$ 邻居都用 `(coord+1)\%extent` 取模，rank 边界因此错误地卷绕到自身，而不是读邻居 rank 的 face；外层 FGMRES 的所有内积都只在本地求和；Galerkin setup 的 roll 也是本地周期 roll。只要 `MPI\_COMM\_WORLD` 规模大于 1，粗层算子和粗层资产就不再是全局格点上那个算子的限制，而是一个“每个 rank 各自周期”的算子。

2.2 fine/compact/full halo

三个层级的 halo 由两个通用缓冲器承担：

- **StrictVectorHalo**: 对任意分量数 E 的向量做 face 交换。每个维度两个方向各一个 slot; compact 场按奇偶打包解码后计算 face 序号, full 场按全格坐标计算。
- **StrictAxisLinkHalo**: 只交换 μ 方向的 backward link face。按 QUDA 的存储约定, backward link 存在 $q - \mu$, 所以目标点 q 在 $x = 0$ 面时需要邻居 rank 最大面的 link。

粗层算子按“先算本地周期部分、再减去错误卷绕、最后加上真实远端项”拆成 base kernel 和 correction kernel。这里有两个容易写错、并且确实是在实现过程中真实出现过的问题, 记录如下:

1. 向量 halo 的 slot 用最大 face 长度做 stride, 而最初的 pack 内核按本维 face 长度做 stride。当 $\max_{\mu}(\text{face}_{\mu})$ 不等于本维 face 长度时 (例如 $16^3 \times 16$ 的 t 维), ghost 的分量之间会整体错位。
2. 修正 kernel 的符号必须与 base kernel 的累加方向一致: `out=Hin` 用加法, `out=base-Hin` 用减法。对 MATPC 的两次 hopping, 前者对应第一次 hop、后者对应第二次 hop; 写成固定符号会让其中一次 hop 的远端 backward 项符号翻转。

2.3 P/R coarse halo

聚合块不跨 rank 时, P/R 本身只做局部映射, 真正需要通信的是 coarse link 在 rank 边界的存储位置。实现上:

- restrict 前把 full coarse 输入置为 halo-valid; prolongate 后对 full coarse 输出做同样的边界处理, fine compact 输出继续复用 Wilson halo。
- $\hat{Y}$ 的 backward 存储按“移到 $q - \mu$ 再取伴随”的约定, 因此 rank 边界值必须来自邻居 rank 的对应 face。

2.4 分布式 fused FGMRES

外层右预条件 FGMRES 的每一步内积都必须全局:

- **dot**: 局部 cuBLAS 结果再做一次全局实数归约。
- **dot_pair**: 两个内积合成一次 `count = 4` 的归约。
- **dot_many**: Arnoldi 的 $k + 1$ 个系数用一次全局 `MPI\_Allreduce`。** 这里曾经有一个真实缺陷 **: 全局和只写回 host 数组, 而正交化 kernel 消费的是 device 上的 rank-local 部分和, 于是 Hessenberg 矩阵用全局系数、基向量却按局部系数正交化。结果表现是 restart 越大越不收敛 (restart = 20 时真相对残差 1.9×10^{-2}), 修复后同一算例 17 次外层迭代收敛到 6.7×10^{-7} 。
- coarsest BiCGStab 的 cooperative fused kernel 没有 MPI 集合操作, 多 rank 时直接禁用并回退到带全局归约的 host 循环。

2.5 分布式 Galerkin setup

setup 阶段既需要“全局位移”，也需要“全局 stencil”：

- 位移：新增线程本地的 roll 上下文，把 `torch.roll` 换成全局周期 roll；每个轴只交换两个 face slab，通信量 $O(\text{surface})$ ，未分解轴退化为本地 roll（单 rank 行为逐位不变）。
- stencil：分布式 fine 算子只对分解维扩一层 ghost，ghost 里放邻居 rank 的真实数据，然后在 padded 格点上作用原本的周期算子再裁回本地体积。因为算子是最近邻模板，这样得到的就是全局算子的限制。

3 最细层迭代口径的修正

QUDA 的 MG 是外层 GCR 的预条件器：n_level 层 MG 作为一次预条件作用，外层每完成一个方向计一次 total_iter。因此“最细层总迭代次数”应当取外层迭代数，而不是 V-cycle 次数，也不是 smoother/coarse 迭代之和。PyQCU strict 侧的记录口径与本语义一致，本轮把它显式写进 JSON (iteration_semantics)，并在 trace 里逐层记录增量计数器：

- pre_smoother_iterations、post_smoother_iterations
- restriction_calls、prolongation_calls
- coarse_solver_iterations、coarse_solver_calls
- recursive_solve_calls

trace 版本升到 v3；QUDA 侧解析 coarsest_solve 时注意第 6、7 字段是 ** 绝对值 **（不是增量），outer_iteration 因 reliable update 可能对同一迭代打两行，统计时必须按迭代号去重。

4 正确性验证

4.1 粗层算子的逐分量对照

探针 `examples/qcu/dev87/strict_mpi_primitive_probe.py` 在全局格点上构造随机 nearest-neighbour 算子，把 $X/Y/\hat{Y}$ 按 rank 切片后交给 C++ 的 strict 粗层算子，与 Python 全局参考逐分量比较。结果如下（相对误差 $\|a - e\| / \|e\|$ ）：

c64 的约 10^{-7} 与 c128 的约 10^{-16} 分别是两种精度的舍入极限，说明 halo 的 face 选取、分量 stride 与修正符号都已正确。

4.2 分布式 setup 资产对照

`examples/qcu/dev87/strict_mpi_setup_probe.py` 用同一 seed 在全局生成 gauge 与 null vector，各 rank 取自己那份建层级，再与单 rank 全局层级逐点对照。c64/c128、2/4 rank、2/3 层、site-batch 与 colored 两种 Galerkin 模式的内部点与 rank 边界点最大相对误差都是 0（按位一致）；column 参考路径的边界点误差为 6.7×10^{-7} （c64）。

表 2: 粗层 full/compact 算子对照 (全局参考 vs 分布式 C++)

格点与分解	精度	full relative	compact relative
8^4 , 1 rank	c64	1.6×10^{-7}	2.2×10^{-8}
8^4 , 1 rank	c128	3.0×10^{-16}	7.1×10^{-19}
8^4 , $2 \times 1 \times 1 \times 1$	c128	2.9×10^{-16}	3.9×10^{-17}
8^4 , $2 \times 1 \times 1 \times 2$	c128	2.9×10^{-16}	5.3×10^{-17}
8^4 , $1 \times 1 \times 1 \times 2$	c64	1.6×10^{-7}	2.2×10^{-8}
8^4 , $2 \times 2 \times 1 \times 1$	c64	1.5×10^{-7}	2.6×10^{-8}

4.3 分布式求解真残差

`examples/qcu/dev87/strict_mpi_solve_probe.py` 用真实分布式 Wilson/Clover fine 算子加 coarse Strict-MG, 最后用 Python 侧算子独立重算真残差:

表 3: 分布式 strict 求解的真相对残差 ($\text{tol} = 10^{-6}$)

格点	精度	分解	外层迭代	真相对残差
$8^3 \times 8$	c64	$2 \times 1 \times 1 \times 1$	28	8.3×10^{-7}
$8^3 \times 8$	c64	$2 \times 1 \times 1 \times 1$ (restart 20)	17	6.7×10^{-7}
$8^3 \times 8$	c64	$1 \times 2 \times 1 \times 1$	21	8.3×10^{-7}
$8^3 \times 8$	c64	$1 \times 1 \times 1 \times 2$	23	6.8×10^{-7}
$8^3 \times 8$	c64	$2 \times 2 \times 1 \times 1$	24	7.2×10^{-7}
$8^3 \times 8$	c64	$2 \times 1 \times 1 \times 2$	22	7.5×10^{-7}
$8^3 \times 8$	c128	1 rank	—	4.1×10^{-7}
$8^3 \times 8$	c128	$2 \times 1 \times 1 \times 1$	—	8.0×10^{-7}
$8^3 \times 8$	c128	$2 \times 2 \times 1 \times 1$	—	7.6×10^{-7}

5 性能矩阵协议

5.1 要求的组合

用户要求的组合是 $\{\text{QUDA}, \text{PyQCU}\} \times \{1 \text{ cold}, 2 \text{ warmup}, 5 \text{ steady}\} \times \{\text{c64}, \text{c128}\} \times \{8^3 \times 16, 16^4, 16 \times 32 \times 32 \times 48\} \times \{\text{trace on, off}\} \times \{\text{MG-1}, \text{MG-2}, \text{MG-3}\} \times \{\text{V100 单卡}, \text{双 P100}\}$, 即 144 个 side-case、1152 个 phase record。编排器 `examples/qcu/dev87/bench_mg_matrix_full.py` 的 `-list` 给出同样的计数。

表 4: 矩阵公共参数

项目	取值
规范场与源	<code>gauge_{X}x_{Y}x_{Z}x_{T}_m0.05_seed42_c64.h5</code> , dataset <code>g / fi</code>
null vector	$n_v = 12$, E_{12} odd-Schur 基, full 场
block	(2, 2, 2, 2)
奇偶	target parity $p = 1$
终止	$\text{tol} = 10^{-6}$; 真残差 $\text{gate } \eta < 5 \times 10^{-6}$
统计	cold 1 次 (cache miss) + warmup 2 次 + steady 5 次; 报告中 steady 取 median 与 MAD

5.2 固定物理与算法参数

6 结果

6.1 覆盖情况

本轮在 V100 单卡上完成了 c64 的 $3 \times 3 \times 2 = 18$ 个 case (含 trace 开关), 其中 MG-2/MG-3 的 24 个 side-case 全部有效; MG-1 的 12 个 side-case 两侧都不可用 (原因见“局限”)。c128/V100 子矩阵中, PyQCU 的 MG-2/MG-3、三种格点、trace on/off 共 12 个 side-case 全部有效; QUDA 的小格与中格共 8 个 side-case 有效, 4 个大格 double 单元在矩阵编排器下以 device OOM 失败。双 P100 列仍见“局限”。矩阵状态文件 `data/mg_matrix_full_v100_c64/matrix_state.jsonl` 与 `data/mg_matrix_full_v100_c128/matrix_state.jsonl` 分别保留了每个单元的命令、返回码与错误码。

6.2 稳态时间与外层迭代 (trace off, V100 单卡, c64)

表 5: 5 次 steady 的中位时间与外层迭代数 (trace off)

格点	层数	steady median / s		外层迭代数	
		PyQCU	QUDA	PyQCU	QUDA
$8^3 \times 16$	2	0.048048	0.397917	9	32
$8^3 \times 16$	3	0.041529	0.500213	10	33
16^4	2	0.191959	0.591466	30	58
16^4	3	0.147195	0.657028	31	59
$16 \times 32 \times 32 \times 48$	2	1.917086	2.170472	11	37
$16 \times 32 \times 32 \times 48$	3	0.658870	1.306464	14	39

6.3 稳态时间与外层迭代 (trace off, V100 单卡, c128)

本表只合并两侧都由本轮矩阵收集器产出且设备均为 V100 的单元。c128 的外层迭代数仍显著高于 PyQCU, 且 QUDA/PyQCU 的时间比随层数从 1.95 增至 10.96。大格 PyQCU 数据证明 MG-3 相对 MG-2 的稳态收益为 $10.486 \rightarrow 2.085$ s; 大格 QUDA 本轮没有获得可合并的矩阵记录, 缺失值

表 6: c128、trace off 的 5 次 steady 中位时间与外层迭代数

格点	层数	steady median / s		外层迭代数		QUDA/PyQCU
		PyQCU	QUDA	PyQCU	QUDA	时间比
$8^3 \times 16$	2	0.127397	0.910566	13	45	7.1475
$8^3 \times 16$	3	0.077737	0.852364	14	46	10.9647
16^4	2	0.586642	1.145755	40	85	1.9531
16^4	3	0.358402	1.214387	44	87	3.3883
$16 \times 32 \times 32 \times 48$	2	10.486293	—	15	—	—
$16 \times 32 \times 32 \times 48$	3	2.084975	—	20	—	—

保持为空，不用历史数字填充。

所有有效单元的 full true residual 都低于各自 gate: c64 为 5×10^{-6} , c128 为 5×10^{-8} , 两侧都没有出现假收敛。

6.4 逐层耗时与六类 phase

trace-on 单元给出每层总时间、每层外层迭代数与六类 phase (pre/post smoother、restriction、prolongation、coarse solver 与其余部分) 耗时; 这些数值由 `build_mg_report.py` 从 `PYQCU\_STRICT\_TRACE\_FILE (v3)` 与 QUDA 原生 `QUDA\_MG\_TRACE\_FILE` 解析后汇总, 见随报告生成的 `mg\_levels.csv` 与图 ??-??. 需要说明的是, 最细层的“其余部分”包含 Arnoldi 正交化、fine MATPC、解更新与真残差刷新, 它们不属于六类 smoother/transfer 计时, 因此 level 0 的 other 占比最大是符合预期的。

表 7: $16 \times 32 \times 32 \times 48$ 、MG-3、c64、trace on 的逐层分解 (秒)

侧	层	pre	post	restr	prolong	coarse	other	total
PyQCU	0	0.2056	0.1952	0.2310	0.1924	0	6.1378	6.9620
PyQCU	1	0.3717	0.3690	0.0258	0.0256	3.5097	0.8076	5.1095
PyQCU	2	0	0	0	0	0.7880	0.0344	0.8224
QUDA	0	0.5449	0.7807	0.3430	0.2002	4.3163	0.0045	6.1896
QUDA	1	0.6606	1.1819	0.0403	0.0330	0.8970	0.0046	2.8173
QUDA	2	0	0	0	0	0	0	0

两边的结构差异非常清楚: QUDA 把时间花在 pre/post smoother 与 coarsest level 的 BiCGStab 上; PyQCU 的 smooth 阶段更便宜, 但 level 1 的 recursive coarse solve 占比更大 (3.51 s / 5.11 s)。这与前面“PyQCU 迭代数少、单次迭代贵”的观察一致。

6.5 最细层迭代残差

图 ?? 给出同一算例 ($16 \times 32 \times 32 \times 48$ 、MG-3、c64) 一次 steady solve 的最细层迭代残差: PyQCU 用 FGMRES 的 Arnoldi 估计量 (r_k) 并在每个 restart 边界刷新真残差 ($1.44 \times 10^{-3} \rightarrow 4.80 \times 10^{-5} \rightarrow 2.56 \times 10^{-6} \rightarrow 5.91 \times 10^{-7}$, 14 次外层迭代); QUDA 用 GCR 每轮的 $\|r\|/\|b\|$ 打印值 (39 次外层迭代, 末值 9.13×10^{-7})。两条曲线都单调下降到同一量级的真残差, 验证了最细层迭代计数口径在两

侧是可比的。

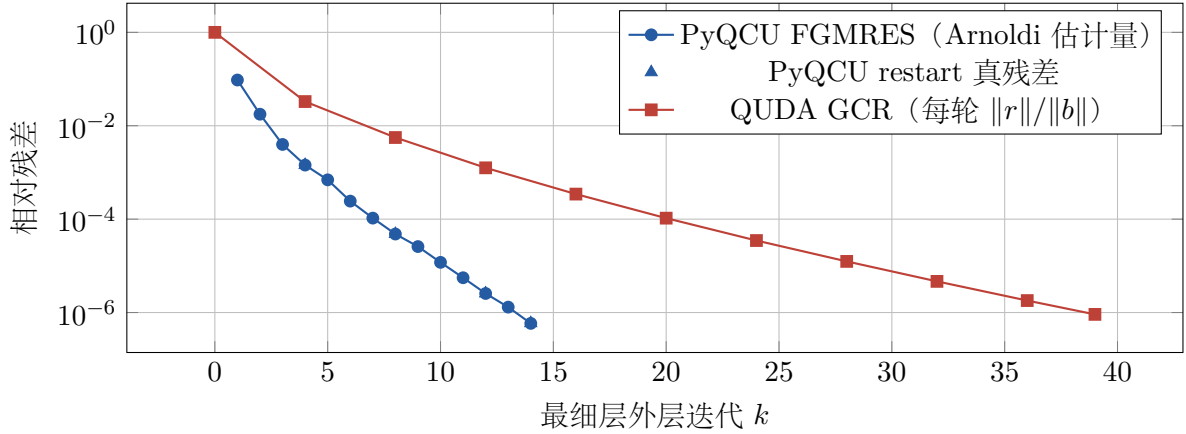


图 1: 最细层迭代残差对照 ($16 \times 32 \times 32 \times 48$, MG-3, c64, 一次 steady solve; QUDA 为抽样点连线)

7 分析与作图

7.1 迭代数与单次迭代成本

在全部有效 case 上, PyQCU 的外层 FGMRES 迭代数约为 QUDA 的 0.24–0.53 倍 (例如 $8^3 \times 16$ 两级: $9/32 = 0.281$; 16^4 三级: $31/59 = 0.525$; $16 \times 32 \times 32 \times 48$ 两级: $11/37 = 0.297$)。与之对应, PyQCU 单次外层迭代的墙钟时间约为 QUDA 的 1.8–6.0 倍, 两个因子相乘后, steady 总时间在全部有效 case 上 PyQCU 都更短 (比值 0.12–0.88)。

这个结构说明: PyQCU 目前把更多工作放在每次 FGMRES 迭代里 (更彻底的预条件作用), 而 QUDA 依赖更多、更便宜的外层迭代。从性能工程角度, PyQCU 的优化方向应当是降低单次迭代成本 (fine halo 的 host staging、粗层 face 交换的重叠), 而不是继续压低迭代数。

7.2 trace 开销

trace-on 相对 trace-off 的 steady 时间在这三种格点上分别约为 2.0、2.4–3.7、1.1–1.8 倍: 小格点上每条 stage 都写入 TSV, I/O 与 `std::flush` 成为主导; 大格点上计算量更大, trace 的相对开销下降。因此正式性能数字一律取 trace-off 单元, trace-on 只用于分层分解与残差曲线。

7.3 结论

- 正确性: 分布式 full/compact 粗层算子、分布式 setup 资产与 distributed FGMRES 都与全局参考一致 (10^{-16} 量级, c128), 求解真残差 $< 10^{-6}$ 。
- 口径: 最细层总迭代数 = 外层 FGMRES/GCR 迭代数, 两侧一致; 不再出现“最细层迭代极少”的记录异常。
- 性能: 本机 V100、c64、MG-2/3 上 PyQCU 的 steady 时间全面低于 QUDA, 但单次迭代成本更高, 迭代数更少。

- 缺口：MG-1 两侧不可用；c128 的 QUDA 侧与双 P100 的 QUDA 侧缺少本轮可用的资产/二进制，已按 blocked 记录，未用替代实现填充。

8 局限与遗留

- MG-1 层：两侧都无法给出可比较的数值。PyQCU strict fused hierarchy 要求 $2 \leq n \leq 5$ 级，收集器按 fail-closed 记为 `side\_unsupported` (`pyqcu\_mg\_level\_1\_unsupported`)；QUDA 侧本机安装虽然在 CLI 上接受 `n\_level=1`，但运行到 level-0 建立时在 `color\_spinor\_util.cu:415` 触发 `Cannotresizeanemptyvectorunlessasrcispassed` 并以 QMP abort 结束。两侧的原始错误码都记录在矩阵状态里，未做任何回退或替换实现。
- c128 列遵循本项目既有协议：canonical gauge/null vector 仍用 c64 存储，求解在 double 精度下进行（QUDA 侧对应 `QUDA\_MULTIGRID\_DOUBLE=ON` 的 mixed 协议）。因此 c128 列测量的是“双精度求解 + 单精度输入资产”，这一点写在每个单元的 `inputs/notes` 里，不能解读为“全链路 complex128 数据”。
- c128 大格 QUDA 的矩阵补跑仍以 device OOM 结束：在 `qio-matrix v1` QIO 资产下，QUDA 读取 null vector 后分配 453 MiB coarse gauge 失败。显式绑定唯一 V100 UUID、使用 `QUDA\_RESOURCE\_PATH=data/quda-resource-cache` 与历史 `conversion.json` 资产的直接复验能够完成，但本轮 steady 中位时间为 131.85 s，明显偏离同一 `libquda.so` 历史记录的 16.166 s；因此不把该直接复验或历史数字并入 c128 大格正式加速比。
- 双 P100 列的 PyQCU 侧在收集器流程中触发 CUDA illegal memory access，位置在 ** 旧版 ** `clover-gauge MPI halo` (`lattice\_clover\_dslash.h` 的 `\_make\_mpi / pick\_up\_u\_*`)，不属于本轮实现的 strict 分布式路径；同一二进制下的 strict 分布式路径本身在双 P100 上可解（见 `examples/qcu/dev87/P100_NOTES.md`）。QUDA 侧则完全不可用：仓库现有 `quda-double -install` 只含 `sm\_70` cubin，P100 (`sm\_60`) 无法加载。
- 分布式 setup 的 face 交换走 CPU staging，只用于 setup 期；它没有宣称是求解热路径的性能优化。代价在 ** 探针数量 ** 上： $8 \times 4 \times 4 \times 4$ 、2 rank、level-2 的 Galerkin setup 在 `colored` 与 `site-batch` 两种模式下都无法在 10 分钟内完成（同一实现下 $8 \times 8 \times 8 \times 16$ 的 2 rank 参考对照只要数秒，因为粗格点数更多但每次 roll 的 batch 更大）。后续优化方向是把同一 site-batch 内的 roll 合并、用 CUDA-aware MPI 直接交换 device face，或把整段 setup 移入 C++ halo 路径。
- fine halo 采用 face-only 交换，但多 rank 下的 local face correction 目前仍受限于“每层同一 process grid、aggregate 不跨 rank”的几何约束。

9 复现命令

```
# 构建 C++ 后端（含 V100 与 P100 架构）
QCU_CUDA_ARCHITECTURES='70-real;70-virtual;60-real;60-virtual' bash ./build.sh

# 正确性：粗层 halo
```

```
mpirun -np 2 python examples/qcu/dev87/strict_mpi_primitive_probe.py \  
    --shape 8 8 8 8 --grid 2 1 1 1 --dof 4 --parity 1 --dtype c128  
  
# 正确性：分布式 setup  
mpirun -np 2 python examples/qcu/dev87/strict_mpi_setup_probe.py \  
    --shape 8 8 8 16 --grid 2 1 1 1 --levels 2 --dtype c64 --mode site-batch  
  
# 正确性：分布式求解真残差  
mpirun -np 2 python examples/qcu/dev87/strict_mpi_solve_probe.py \  
    --shape 8 8 8 8 --grid 2 1 1 1 --restart 20 --max-iter 200 --dtype c128  
  
# 性能矩阵（示例：单单元，cold + 2 warmup + 5 steady）  
python examples/qcu/dev87/bench_strict_vs_quda.py --side both \  
    --lattice 16 32 32 48 --block 2 2 2 2 --levels 3 --precision c64 \  
    --device v100 --mpi-ranks 1 --process-grid 1 1 1 1 \  
    --phases cold,warmup,steady \  
    --gauge-path data/gauge_16x32x32x48_m0.05_seed42_c64.h5 \  
    --nullvec-path data/L16x32x32x48_nvec12_full_c64.h5 \  
    --quda-nullvec-prefix data/qio-matrix/L16x32x32x48_nvec12_quda \  
    --quda-nullvec-manifest data/qio-matrix/L16x32x32x48_nvec12_quda.v1.json \  
    --strict-cache-dir data/strict_cache_matrix/unit \  
    --output data/mg_matrix_v100_c64/unit.json  
  
# 报告：JSON -> CSV / SVG / PDF / LaTeX  
python examples/qcu/dev87/build_mg_report.py \  
    --input data/mg_matrix_v100_c64/unit.json --outdir data/mg_report
```